

UNIVERSITY OF ESSEX ONLINE

MASTER OF SCIENCE THESIS

Measuring Technical Debt in Mission Critical Trading Systems

A Service Level Quantitative Framework

Author:
Tobias Zeier

Supervisor:
Dr Zahid Ullah

Co-Supervisor:
Doug Millward

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science*

in the

Enterprise IT Management Programme
Computing Department



21 April 2026

Declaration of Authorship

I, Tobias Zeier, declare that this thesis titled *Measuring Technical Debt in Mission Critical Trading Systems*, and the work presented in it are my own. I confirm that:

- This work was done wholly during the course of the computing project module as part of my MSc Enterprise IT Management degree at the University of Essex Online.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.

Signed:

Date:

UNIVERSITY OF ESSEX ONLINE

Abstract

Computing Department

Master of Science

Measuring Technical Debt in Mission Critical Trading Systems

by Tobias Zeier

This thesis develops a service-level quantitative framework for measuring and prioritising technical debt in mission-critical trading systems. Existing measurement approaches rely predominantly on static code analysis, which is inapplicable to vendor-dominated enterprise environments where operational risks manifest in service instability, patch delay and infrastructure obsolescence rather than source code. The framework applies TOPSIS to four operational indicators, incident frequency, mean time to restore, change failure rate and patch recency, derived from live ITSM and CMDB records at a Swiss securities trading institution. Indicator selection is grounded in a critical review of technical debt theory, measurement frameworks, financial enterprise IT governance, and multi-criteria decision-making. The study adopts a Design Science Research paradigm and a pragmatist stance, treating the relationship between degraded operational indicators and technical debt severity as a testable hypothesis rather than an established premise. The framework's evaluation centres on ranking stability under criterion weight perturbation, contributing a reproducible prioritisation instrument that bridges IT service management and software quality engineering in a domain currently underserved by the technical debt literature.

Acknowledgements

I would like to express my sincere gratitude to Dr Zahid Ullah for his expert guidance, patience, and continued support throughout this research project. His insightful feedback and encouragement were invaluable in shaping this work.

I am equally grateful to Doug Millward for his thoughtful co-supervision during the early stages of this thesis and his practical perspectives on the subject matter.

I would also like to thank the University of Essex Online for providing an excellent academic environment and the resources necessary to complete this degree.

Finally, I extend my heartfelt thanks to my colleagues, friends, and family for their understanding and encouragement throughout this journey.

Table of contents

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Aims and Objectives	2
1.4 Research Questions	3
1.5 Significance of the Study	3
1.6 Thesis Structure	3
2 Literature Review	5
2.1 Context of the Topic	5
2.2 Methodology for Literature Selection	6
2.3 Conceptual Foundations of Technical Debt	8
2.4 Measurement and Prioritisation Frameworks	11
2.5 Technical Debt in Financial and Regulated Enterprise IT	13
2.6 Multi-Criteria Decision-Making and TOPSIS	18
2.7 Transferability Critique and Ethical Dimensions	20
2.8 Synthesis: Converging Evidence and the Research Gap	21
3 Research Design and Methodology	24
3.1 Research Design	24
3.2 Ethical and Professional Considerations	32
3.3 Rework!	33
4 Framework Development and Results	34
4.1 Data Preparation	34
4.2 Indicator Normalisation and Weighting	36
4.3 TOPSIS Computation and Rankings	37
4.4 Sensitivity Analysis	37
4.5 Interpretation and Discussion	37
5 Evaluation	38
5.1 Evaluation Aim	38
5.2 Evaluation Criteria	38
5.3 Stability Results	38

5.4	Practical Validity	38
5.5	Limitations of the Evaluation	38
6	Conclusion	39
6.1	Research Summary	39
6.2	Findings Against Research Questions	39
6.3	Contribution to Knowledge	39
6.4	Practical Contribution	39
6.5	Limitations and Future Work	39
6.6	Reflective Note	39
	References	40
	Appendices	49
A	Appendix A	49

List of Figures

3.1	DSR Grid for the present study, adapted from vom Brocke, Hevner and Maedche (2020)	26
3.2	Framework pipeline: from operational indicators to ranked prioritisation	31
3.3	Proposed framework pipeline	33

List of Tables

2.1	Search terms used for literature review	6
2.2	Inclusion and exclusion criteria	7
2.3	Mapping of operational indicators to technical debt dimensions and literature justification	16
3.1	DSR guidelines and their implementation	25
3.2	Operational definition of indicators used in the TOPSIS framework	28
4.1	Raw decision matrix	35
4.2	Weight configurations	37

List of Abbreviations

AHP	Analytic Hierarchy Process
BCS	British Computer Society
CFR	Change Failure Rate
CI/CD	Continuous Integration / Continuous Delivery
CMDB	Configuration Management Database
COTS	Commercial Off the Shelf
DORA	DevOps Research and Assessment
DSR	Design Science Research
FINMA	Swiss Financial Market Supervisory Authority
FADP	Federal Act on Data Protection
IT	Information Technology
ITIL	IT Infrastructure Library
ITSM	IT Service Management
MCDM	Multi-Criteria Decision-Making
MTTR	Mean Time to Restore
P1-3	Priority 1-3
RQ	Research Question
SLA	Service Level Agreement
SQALE	Software Quality Assessment based on Lifecycle Expectations
TD	Technical Debt
TOPSIS	Technique for Order of Preference by Similarity to Ideal Solution
VIKOR	VlseKriterijumska Optimizacija I Kompromisno Resenje

List of Symbols

C_i	TOPSIS closeness coefficient
S_i^+	Euclidean distance from positive ideal solution
S_i^-	Euclidean distance from negative ideal solution
x_{ij}	Raw indicator value for service i , criterion j
r_{ij}	Vector-normalised indicator value
v_{ij}	Weighted normalised indicator value
w_j	Criterion weight
m	Number of services evaluated
n	Number of indicators

1 Introduction

This chapter establishes the research context, defines the problem the study addresses, and sets out the aims, objectives, research questions, and significance of the work. It follows the structure of the thesis as a whole, moving from the broad territory of technical debt in enterprise IT to the specific gap in knowledge that the proposed framework is designed to fill.

1.1 Background and Motivation

Technical debt describes the implied future cost of accepting short-term technical compromises in place of more robust solutions (Cunningham, 1992). Since its introduction, the concept has expanded from source code quality to encompass architectural, design, test and documentation dimensions of complex IT systems (Kruchten, Nord and Ozkaya, 2012). Its operational and financial consequences are measurable. Banker, Liang and Ramasubbu (2020) found that technical debt significantly reduced firm performance over an eleven-year product lifecycle. The penalty more than doubled as debt compounded, and firms with median debt levels saw their system's net value turn negative by year five. In financial trading environments, these consequences are intensified by continuous availability requirements, vendor-dominated architectures and regulatory oversight. This makes the absence of a quantitative prioritisation mechanism particularly costly (Ramasubbu and Kemerer, 2021; Haki *et al.*, 2023).

1.2 Problem Statement

The technical debt literature lacks a standardised, quantitative framework for measuring and prioritising debt at the service level in vendor-dominated enterprise IT. Lenarduzzi *et al.* (2021) reviewed 44 primary studies on prioritisation

and found that quantitative multi-criteria approaches are uncommon and empirical validation on industrial datasets is rare. Dominant measurement tools operationalise debt as remediation effort derived from source code rule violations (Biazotto *et al.*, 2024), rendering them inapplicable where source code access is unavailable, as is typical in trading IT. Even where code is accessible, tools applied to the same codebase produce substantially divergent estimates (Amanatidis *et al.*, 2020), leaving infrastructure lifecycle debt, platform obsolescence and vulnerability debt largely unaddressed. Although DORA metrics have been validated as predictors of software delivery performance (Forsgren, Humble and Kim, 2018), the inferential step from degraded operational indicators to elevated technical debt severity is not yet empirically established and is treated in this study as a testable hypothesis. To the best of current knowledge, no existing framework applies multi-criteria decision-making to live operational service management data in order to produce a ranked technical debt prioritisation instrument for financial trading IT. This gap defines the research problem.

1.3 Research Aims and Objectives

This study develops a service-level quantitative framework for measuring and prioritising technical debt in mission-critical trading systems, grounded in operational service management data rather than source code analysis. The specific objectives are as follows.

1. To review literature on technical debt measurement, prioritisation, financial enterprise IT governance and multi-criteria decision-making.
2. To identify and operationally define quantifiable technical debt indicators derived from live ITSM and CMDB records.
3. To design and apply a TOPSIS-based multi-criteria model for scoring and ranking services by technical debt priority.
4. To evaluate the framework assessing ranking stability under criterion weight perturbation.

1.4 Research Questions

The study is guided by the following primary research question.

RQ1: *How can technical debt in mission-critical trading systems be measured quantitatively at the service level?*

Two secondary questions support the investigation.

- RQ2: To what extent do operational service management metrics serve as valid proxies for technical debt severity in vendor-dominated enterprise IT environments?
- RQ3: How stable are the resulting service rankings when criterion weights are perturbed?

1.5 Significance of the Study

The principal academic contribution is a prescriptive design artefact that bridges IT service management and software quality engineering in a domain currently underserved by the technical debt literature. To the best of current knowledge, no prior study has applied TOPSIS to live operational records from a financial trading institution for technical debt prioritisation. Practically, the framework operates on data routinely available in enterprise ITSM and CMDB systems without requiring source code access, making it directly applicable to vendor-dominated trading IT architectures and supporting auditable remediation decisions in regulated environments (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024).

1.6 Thesis Structure

Chapter 2 reviews the theoretical and empirical foundations across four bodies of literature and identifies the research gap. Chapter 3 describes the research design, methodology and ethical considerations. Chapter 4 presents the framework development and results, including TOPSIS computation and sensitivity analysis. Chapter 5 evaluates the framework against the criteria established in Chapter 3. Chapter 6 concludes with responses to the research questions, a

critical self-evaluation, the academic and practical contributions, and directions for future work.

2 Literature Review

This chapter develops the theoretical and empirical foundation for the proposed framework through a critical review of four interconnected bodies of literature: the conceptual foundations of technical debt, existing measurement and prioritisation approaches, technical debt governance in financial and regulated enterprise IT, and multi-criteria decision-making. Each section synthesises convergent findings, identifies tensions and limitations, and positions these gaps in relation to the research design. The review adopts a critical, problem-oriented perspective rather than an exhaustive catalogue, building a coherent argument that existing approaches are not adequate for service-level enterprise IT contexts and that a quantitative framework applying TOPSIS to operational service management data represents a defensible and novel contribution.

2.1 Context of the Topic

Technical debt has become both a central concept and a persistent management problem in software engineering and enterprise IT governance (Avgeriou *et al.*, 2023). Ward Cunningham first introduced the metaphor in 1992, describing technical debt as the long-term cost of short-term technical compromises (Cunningham, 1992). Over time, the concept has expanded from a focus on poor code to a broader understanding that also includes architectural, infrastructure, organisational and process aspects of complex IT systems (Kruchten, Nord and Ozkaya, 2012).

Despite this conceptual expansion, much of the empirical and tool-supported literature remains anchored at the source code level, relying heavily on static analysis tools and code-based metrics. These approaches work tolerably well for greenfield development dominated by in-house code but are much less appropriate for enterprise environments where operational reliability depends

on vendor platforms, legacy infrastructure and tightly integrated service stacks (Amanatidis *et al.*, 2020).

The present review is motivated by a gap at the intersection of technical debt management, multi-criteria decision-making and mission-critical enterprise IT operations. To the best of current literature, no framework provides a quantitative, service-level mechanism for measuring and prioritising technical debt in environments such as trading IT. In these contexts, significant forms of technical debt manifest not only in code repositories but also in operational outcomes such as service instability, patch delay and infrastructure obsolescence (Ramasubbu and Kemerer, 2021; Haki *et al.*, 2023). The following sections build a theoretical and methodological basis for the proposed framework.

2.2 Methodology for Literature Selection

The literature search was conducted between January and April 2026 using a systematic and reproducible approach, drawing on four principal academic databases: the University of Essex library, Scopus, IEEE Xplore and Google Scholar. Search terms included the following.

TABLE 2.1: Search terms used for literature review

Theme	Search String
TD foundations	"technical debt" AND (metaphor OR taxonomy OR typology OR theory)
TD measurement and tools	"technical debt" AND (measurement OR "static analysis" OR SonarQube OR metric OR "code smell")
TD prioritisation	"technical debt" AND (priorit* OR "backlog management" OR remediation OR repayment)
TD in enterprise and financial IT	"technical debt" AND ("enterprise" OR "financial services" OR "trading" OR "legacy system" OR "regulated")
TD automation	"technical debt management" AND ("machine learning" OR intelligent OR automation OR NLP)

Theme	Search String
MCDM and TOPSIS	TOPSIS AND ("multi-criteria" OR MCDM OR "SAW" OR "decision making" OR priorit*)
Patch management	"patch management" AND (priorit* OR risk OR vulnerability OR SLA OR enterprise)
Operational risk and DORA	("DORA metrics" OR "mean time to restore") AND ("technical debt" OR "software delivery")

Sources were screened against the following pre-specified inclusion and exclusion criteria:

TABLE 2.2: Inclusion and exclusion criteria

Criterion	Rule
Publication type	Peer-reviewed journal articles, conference proceedings or empirical technical reports
Language	English full text available
Date range	2019–2026, or foundational regardless of date
Thematic relevance	Directly addresses at least one of the four thematic pillars
Exclusion: scope	Opinion pieces without empirical grounding
Exclusion: topic	Agile process improvement without a measurable TD component
Exclusion: duplicates	Superseded by a more comprehensive journal version

Screening was conducted in two stages. Titles and abstracts of all retrieved records were first assessed against these criteria, reducing the initial pool. Remaining sources then underwent full-text review to verify relevance and methodological quality. Forward and backward citation snowballing was subsequently applied to three anchor publications: Kruchten, Nord and Ozkaya (2012), Lenarduzzi *et al.* (2021) and Forsgren, Humble and Kim (2018) to surface relevant studies not captured by database searches. The process yielded approximately 110 candidate sources, of which more than 50 were retained following full-text screening.

The review uses narrative synthesis rather than a formal systematic review protocol. The research spans three disciplines, namely software engineering, enterprise IT governance and decision science, which differ significantly in their research methods, terminology and evidence standards. A single meta-analytic framework would not accommodate this heterogeneity, and the aim of the review is to build a coherent argument in support of a novel design science artefact, for which narrative synthesis is the appropriate approach (Hevner *et al.*, 2004). The following four sections build the theoretical and contextual case for the proposed framework in sequence.

2.3 Conceptual Foundations of Technical Debt

The first thematic area traces the evolution of the technical debt concept from the original metaphor to later theoretical developments, and then to its expansion beyond software into systems engineering, infrastructure and vendor platforms. Understanding this evolution is essential because most existing tools and practices are still tuned to an earlier, narrower view of technical debt that misaligns with the realities of contemporary enterprise IT.

2.3.1 Origins and the Metaphor

Cunningham (1992) introduced the metaphor to explain his refactoring strategy to non-technical stakeholders. In his original formulation, shipping code that is “not quite right” resembles taking on financial debt, with each subsequent maintenance effort representing interest. If the principal is not repaid through refactoring, the cost of working with the system compounds until change becomes very difficult.

The debt metaphor has proved productive, but it carries important conceptual limits. In financial debt, the principal is contractually defined, and the interest rate is known in advance, whereas technical debt has neither. The principal is an estimated remediation cost that changes as systems evolve. The interest depends on future development activity that is uncertain at the time of incurrence. These limitations matter directly for the present research because a central objective of the proposed framework is to make interest cost concrete and operationally

measurable. The framework therefore does not rely on the metaphor for rigour; it grounds its estimates in service-level operational data rather than in abstract financial analogies.

Several influential practitioners refined the concept. McConnell (2008) introduced a cost structure distinguishing principal, recurring interest and compounding interest, and differentiated unintentional debt from intentional debt incurred as a conscious trade-off. Fowler (2009) proposed a quadrant distinguishing reckless from prudent debt and deliberate from inadvertent debt. It should be noted that the quadrant originates in a practitioner blog post rather than a peer-reviewed publication; it is a useful heuristic for stakeholder communication but should not be treated as a theoretically validated framework. Kruchten, Nord and Ozkaya (2012) subsequently formalised a taxonomy covering code, design, architectural, test, build and documentation debt on the basis of a structured research programme, and this taxonomy underpins much of the subsequent empirical work.

2.3.2 Theoretical Development

Empirical studies have confirmed and extended this view. Besker (2020) reported that developers in industrial settings spend roughly 23% of their time dealing with the consequences of existing technical debt, with architectural debt exerting the strongest negative influence on productivity. Bedi and Kaur (2022) used regression analysis on open-source repositories and found that commit frequency, lines of code, code smells, and test coverage are significant predictors of technical debt, whereas developer reputation metrics are not. This suggests that technical debt is driven primarily by development practices rather than by the perceived quality of individual contributors.

Ahmad *et al.* (2026) approached the topic qualitatively, interviewing experts from four international companies. They identified five non-technical dimensions of debt accumulation in large-scale agile environments, namely social dynamics, process, people, documentation and requirements, with lack of communication emerging as the dominant driver. This reinforces the view of

technical debt as a sociotechnical phenomenon that cannot be governed by technical measurement alone.

Vidoni, Codabux and Fard (2022) advanced a more radical argument through a game-theoretic lens, proposing that in complex systems technical debt cannot be completely removed and that its effects cannot be fully controlled even when individual items are repaid. The concept of “infinite technical debt” challenges the assumption, implicit in many repayment-focused frameworks, that debt is always recoverable given sufficient effort. This argument has not yet been empirically validated in enterprise IT settings and should be understood as a theoretical provocation rather than an established finding. Its practical implication for the present framework is that governance outputs should surface replacement decisions where service-level data suggests debt has crossed a threshold beyond which incremental repayment is unlikely to be effective.

2.3.3 Expanding the Conceptual Perimeter

The conceptual perimeter of technical debt has progressively widened to include broader systems and lifecycle concerns. Kleinwaks, Batchelor and Bradley (2023) conducted a systematic review of technical debt in systems engineering and found that requirements, architecture, integration artefacts and verification artefacts can all accumulate debt, and that many software-oriented approaches are poorly adapted to multidisciplinary systems engineering lifecycles. Hanssen, Brataas and Martini (2019) identified scalability debt as a distinct subtype triggered by regulatory change: in an open banking case study, they showed that architectural decisions, adequate before the EU Payment Services Directive 2 (PSD2), became sources of debt once new scalability requirements were imposed without architectural review. This illustrates how external regulatory change can convert previously adequate designs into liabilities, a pattern directly relevant to financial trading IT where regulatory obligations are similarly dynamic. Yang *et al.* (2019) proposed a taxonomy of Commercial Off-the-Shelf (COTS) related technical debt covering seven key types, including vendor platform obsolescence and integration constraints. Managing such forms of debt requires different tools from those used for internally developed code. For financial trading IT, where vendor platforms and COTS products are central, this broader

view is particularly important: many of the significant forms of technical debts, including platform debt, infrastructure debt and vulnerability debt, are not detectable by standard static analysis.

2.4 Measurement and Prioritisation Frameworks

The second thematic area examines how technical debt is measured and prioritised in practice. It argues that current measurement is heavily concentrated at the code level and that existing prioritisation frameworks lack the multi-dimensional quantitative rigour required for robust governance in enterprise settings.

2.4.1 The Dominance of Static Analysis and Its Limits

Static code analysis tools such as SonarQube, SQALE and CAST dominate current practice, typically operationalising technical debt as a weighted sum of estimated remediation effort derived from rule violations (Biazotto *et al.*, 2024). In a systematic mapping study, Biazotto *et al.* (2024) identified 178 primary studies and found that most automation artefacts operate at the code level, that substantial human intervention is still required even at higher automation levels, and that end-to-end automation across the technical debt management lifecycle is rare. The focus on code leaves infrastructure-level and platform-level debt largely unaddressed. Ampatzoglou *et al.* (2022) presented SDK4ED as a comprehensive industry oriented platform integrating several code level techniques. A three-year evaluation suggested that SDK4ED can deliver financial benefits to software organisations. At the same time, the platform offers only limited support for debt related to infrastructure or vendor platforms. Tornhill and Borg (2022) analysed 39 proprietary production codebases and found that poor code quality is statistically associated with higher defect density, longer remediation cycles and reduced developer throughput, confirming that code-level debt has clear business consequences. This does not change the fact that an exclusive focus on code creates blind spots: organisations may allocate significant resources to refactoring while leaving critical legacy infrastructure and unsupported components unaddressed.

Amanatidis *et al.* (2020) evaluated agreement between different technical debt assessment tools applied to 50 open-source projects and observed substantial divergence: tools applied to the same codebase produce different estimates and highlight different violation sets, with no single tool consistently dominating. This implies that a debt figure generated by any one tool is a function of that tool's ruleset and thresholds rather than an objective measurement. Albuquerque *et al.* (2023) reinforced this finding at scale, noting that infrastructure debt and process debt remain significantly understudied compared to code and architectural debt.

2.4.2 The Prioritisation Gap

Lenarduzzi *et al.* (2021) carried out the most comprehensive review of technical debt prioritisation strategies to date, analysing 44 primary studies. They found that existing approaches cluster around cost-based, value-based and risk-based strategies, that most adopt a single primary criterion, that quantitative methods are less common than heuristic techniques, and that empirical validation on industrial datasets is rare. They concluded that the field lacks agreement on which factors matter, how to measure them and what constitutes an adequate prioritisation process. This conclusion directly motivates the quantitative contribution of the present work. The stakes of inadequate prioritisation are concrete: de Toledo *et al.* (2021) showed in a production microservices system that targeted repayment of architectural technical debt led to an 84% reduction in incidents, anchoring the case for prioritisation in a measurable operational outcome.

Empirical studies consistently reveal a gap between tool-generated priorities and practitioner behaviour. Practitioners consider both value and cost rather than a single strategy, and perceived value is highly context-dependent (Alfayez *et al.*, 2023). Pina, Seaman and Goldman (2022) surveyed developers and found that prioritisation decisions reflect a combination of technical severity, business impact and available effort, and that developers frequently experience tension between technically optimal repayment choices and organisationally imposed constraints. Most IT managers nonetheless lack a formal technical debt management process, with cross-functional communication identified as the

primary obstacle (Wiese and Borowa, 2023). Stochel *et al.* (2023) proposed the Continuous Debt Valuation Approach (CoDVA), which assigns a financial value to each debt item based on remediation cost, interest rate and business risk exposure; case studies show that financial valuation can significantly change repayment priorities relative to effort-only approaches, but CoDVA still relies on development-level data and does not incorporate operational service management signals such as incident rates or change failure rates.

These studies collectively indicate that any quantitative prioritisation framework embedding value judgements about which systems and risks matter most, must make those judgements transparent and auditable. Particularly in regulated institutions, where priority scores may influence funding, staffing and post-incident accountability.

2.5 Technical Debt in Financial and Regulated Enterprise IT

The third thematic area applies these general insights to the specific setting of financial services and regulated trading IT, focusing on how debt manifests operationally, how it translates into measurable harm, and how related work in security and patch management quantifies that harm.

2.5.1 The Financial Services Context

Financial services organisations are characterised by complex, layered legacies, strict regulatory compliance requirements and stringent service availability obligations, with architectures that often depend on vendor platforms and COTS products that cannot be freely refactored (Haki *et al.*, 2023).

Haki *et al.* (2023) conducted clinical research at Credit Suisse, designing and introducing a digital nudge for technical debt management across more than 3,000 IT applications. As part of this work, they defined a technical debt taxonomy with four main categories: code quality debt, infrastructure lifecycle debt, vulnerability debt and automation debt. Infrastructure lifecycle debt was measured as cumulative months of unsupported components; vulnerability

debt was derived from weighted counts of scanner-detected vulnerabilities. This taxonomy is significant for two reasons. First, it shows that a major financial institution assigns equal operational importance to infrastructure and vulnerability debt as to code-level debt. Second, this directly supports the broader measurement approach taken in the present research.

The empirical basis of the evaluation is, however, limited. The quantitative analysis focused on three development teams, two of which reduced their debt index over one year while the third did not. The authors identified management intent as the decisive factor, rather than the nudge design itself. The clinical research design, with researchers embedded in the organisation, also raises questions about independence and replicability. These factors do not invalidate the conceptual contribution, but they caution against generalising from the observed outcomes to other institutions or debt types.

Rolland and Lyytinen (2021) investigated the organisational dynamics of architectural debt accumulation at a large Scandinavian financial institution through eleven in-depth interviews with business managers, enterprise architects and developers. Their analysis identified two persistent tensions: between decentralised feature delivery teams optimising for local throughput and the governance required to manage debt at the system level, and between the pace of digital innovation and the long-term sustainability of the supporting architecture. Both tensions are intensified in financial services by compliance obligations and competitive pressure to release new products rapidly. The study is notable for grounding these dynamics in empirical data from a regulated banking institution. Its primary limitation is the single-case design; whether the patterns observed generalise to institutions with different architectural profiles, including trading-specific environments, remains an open question.

2.5.2 Enterprise Systems and the Outsourcing Context

Banker, Liang and Ramasubbu (2020) examined 26 firms operating a commercial COTS CRM system over its full eleven-year lifecycle. Using propensity score matching against firms with zero technical debt, they found that a 10% increase in technical debt reduced gross return on assets by 16% on average. Critically,

this penalty more than doubled within six years of adoption as code decay compounded structural complexity. While all firms initially benefited from system customisation, those carrying median debt levels saw the net value of their system turn negative by year five, a pattern robust to alternative performance measures including asset turnover, gross margin and operating income.

In a later study, Ramasubbu and Kemerer (2021) extended this analysis to 1,824 outsourced remediation projects involving 424 client organisations. They found that “control balancing” between client and vendor improved remediation outcomes only when both parties had agreed a long-term migration roadmap. In the absence of such a roadmap, increased coordination actually harmed performance. For prioritisation frameworks, this implies that ranking debt items without embedding them in a strategic roadmap can be counterproductive.

Rinta-Kahila *et al.* (2023) examined a global manufacturing firm’s COTS implementation and showed how staged legacy replacement, combined with recurrent customisations to restore local flexibility, led to self-reinforcing architectural debt accumulation. A system dynamics model captured reinforcing feedback loops between customisation decisions, debt accumulation and declining digital options. This work suggests that organisations do not typically neglect technical debt deliberately. Instead, they are trapped by interactions between technical design choices and organisational routines that standard code-level tools cannot reveal. Governance-level visibility of infrastructure and architectural debt is required to detect these dynamics before they become irreversible.

2.5.3 DORA Metrics as Operational Debt Proxies

Forsgren, Humble and Kim (2018) validated DORA metrics across more than 2,000 organisations, showing that MTTR, CFR, deployment frequency and lead time for changes distinguish high-performing from low-performing delivery teams and can be derived from ITSM tools without source code access. It is important to acknowledge that DORA metrics were validated as delivery performance predictors, not as technical debt proxies. The inferential step from degraded indicators to elevated TD severity is plausible but not empirically

established as a general relationship. The present framework therefore treats this as a testable hypothesis rather than a given premise. Nord, Ozkaya and Woody (2021) provide more direct grounding by classifying delivery tempo degradation and deployment instability explicitly as observable TD symptoms detectable from production records, positioning MTTR and CFR as diagnostic signals of accumulated debt rather than borrowed performance surrogates.

The evidence nonetheless requires careful qualification. Paudel *et al.* (2025) found that TD density explained between 5% and 41% of lead time variance across six fintech components, with team structure and component ownership acting as confounding factors. This reinforces the case for multi-criteria measurement rather than reliance on any single proxy. Nielsen and Skaarup (2022) offer complementary empirical support. In a vendor-managed public sector portfolio without in-house development resources, operational errors emerged shortly after TD-related resource constraints were imposed on a shared service component, and unsupported component versions were only detected through infrastructure migration rather than routine monitoring. While the public sector context limits direct transferability, the operational mechanism is consistent with the vendor-dominated profile of the case organisation.

The following table maps each operational indicator to the technical debt dimension it proxies and the literature that justifies its inclusion:

TABLE 2.3: Mapping of operational indicators to technical debt dimensions and literature justification

Indicator	TD Dimension	Literature Justification
Incident frequency	Architectural debt	de Toledo <i>et al.</i> (2021); Ramasubbu and Kemerer (2021); Ramasubbu, Kemerer and Woodard (2015); Nielsen and Skaarup (2022)
Mean Time to Restore (MTTR)	Infrastructure / platform debt	Forsgren, Humble and Kim (2018); Nord, Ozkaya and Woody (2021); Paudel <i>et al.</i> (2025)

Indicator	TD Dimension	Literature Justification
Change failure rate	Process debt	Forsgren, Humble and Kim (2018); Nord, Ozkaya and Woody (2021); Paudel <i>et al.</i> (2025)
Patch recency	Vulnerability debt	Tizio, Armellini and Massacci (2023); Markkanen and Frantti (2023)
Unsupported component months	Infrastructure lifecycle debt	Haki <i>et al.</i> (2023)

Using operational service management data as a basis for technical debt prioritisation raises important ethical questions, addressed in full in Section 3.2.

2.5.4 Security Debt and Patch Management

Security-related technical debt is most visible in the literature on patch management and vulnerability handling. Tizio, Armellini and Massacci (2023) analysed 86 Advanced Persistent Threat campaigns dated between 2008 and 2020, finding that enterprises delaying updates by one month faced 4.9 times higher odds of compromise compared to immediate patching, rising to 9.1 times at three months. Contrary to common assumption, most campaigns exploited publicly known vulnerabilities rather than zero-days, confirming that unpatched systems carry measurable, quantifiable risk. These figures should be contextualised carefully: the dataset spans all industries and predates regulatory changes, including FINMA circular 2023/1 and the EU DORA regulation, that have since accelerated patching cycles in financial services specifically (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024). The risk multipliers are therefore best treated as indicative of the direction and order of magnitude of the relationship rather than as directly applicable benchmarks for contemporary trading IT.

Markkanen and Frantti (2023) reviewed current NIST guidance and the academic literature on patch management planning. They argue that uniform,

time-based patching policies are increasingly inadequate given the shrinking window between vulnerability disclosure and exploitation, and that organisations should treat patching as a risk-based, organisation-wide business priority rather than a purely technical responsibility. Nurse (2025) framed patch prioritisation as a problem of maintaining SLA compliance, noting that business partners and clients drive patching decisions through supply chain agreements and contractual requirements. This aligns patch management directly with the service-level focus of the present research.

Kula *et al.* (2019) studied release practices and dependency management at ING, showing that keeping upstream dependencies current is a persistent challenge in rapid-release banking environments and that delays in dependency updates contribute directly to technical debt. This finding is consistent with the broader argument, that technical debt manifests in ways detectable through operational service management data. The study is based on a single-organisation case study, however, and may reflect ING-specific governance practices; whether the same dependency dynamics apply in trading IT environments with different release governance structures is not established.

Together, these studies illustrate that in financial trading IT, significant forms of technical debt extend well beyond source code. Regulatory change can convert previously adequate architectures into sources of debt overnight (Hanssen, Brataas and Martini, 2019), unsupported components accumulate measurable infrastructure lifecycle debt (Haki *et al.*, 2023), and service instability signals accumulated debt that compounds with system growth (Ramasubbu and Kemerer, 2021). Each of these manifestations is detectable through ITSM and CMDB records rather than static analysis, establishing the data foundation for the proposed framework.

2.6 Multi-Criteria Decision-Making and TOPSIS

The fourth thematic area argues that multi-criteria decision-making (MCDM) methods are well suited to the technical debt prioritisation problem and that TOPSIS is more appropriate for this context than simpler alternatives.

2.6.1 The Case for Multi-Criteria Methods

Technical debt prioritisation involves balancing several criteria simultaneously: remediation cost, business value, security risk, operational impact and regulatory compliance. Practitioners weigh these dimensions together rather than applying a single factor (Alfayez *et al.*, 2023). Lenarduzzi *et al.* (2021) concluded that the absence of a validated, quantitative, multi-criteria framework is a central obstacle to evidence-based technical debt governance.

A weighted scoring matrix is the simplest alternative, but it assumes that criteria are independent and commensurable across scales, neither of which holds reliably when combining continuous cost metrics with ordinal or binary compliance indicators. A risk matrix is categorical and does not support fine-grained ranking of a large service portfolio. TOPSIS addresses both limitations by normalising heterogeneous criteria before aggregation and ranking alternatives by their geometric proximity to the ideal solution, making it well suited to contexts where criteria differ in type, scale and direction of preference (Ciardiello and Genovese, 2023).

2.6.2 TOPSIS: Mechanics, Applications and Critical Limitations

Hwang and Yoon (1981) introduced TOPSIS, defining the ideal solution as the hypothetical alternative scoring best on all criteria simultaneously and ranking alternatives by Euclidean proximity to this ideal. Zanakakis *et al.* (1998) showed that TOPSIS produces stable rankings close to the Pareto-optimal ordering across a range of problem structures, supporting its use in prioritisation tasks under uncertainty.

TOPSIS constructs a normalised weighted decision matrix, identifies ideal best and worst values per criterion, and ranks alternatives by their closeness coefficient, the ratio of the distance to the ideal worst to the sum of both distances (Albarak *et al.*, 2022). A well-known limitation is rank reversal: adding or removing alternatives can shift the rankings of existing options (García-Cascales and Lamata, 2012). Yang (2020) proposed a normalisation adjustment that mitigates this by defining reference points independently of the current

alternative set, which is particularly relevant for enterprise portfolios where services are regularly added or decommissioned.

Three alternative MCDM methods warrant comparison. VIKOR identifies compromise solutions between conflicting criteria rather than producing a global ranking, making it less appropriate when the output required is a ranked remediation list (Shekhovtsov and Saabun, 2020). AHP derives weights through pairwise comparisons but does not natively produce a full portfolio ranking, and the comparison process grows burdensome as the number of criteria increases (Ishak and Wanli, 2020). A weighted scoring matrix is simpler still but assumes criteria are measured on comparable scales, which does not hold when mixing continuous operational metrics with categorical compliance indicators. TOPSIS integrates normalisation, weighting and ranking into a single procedure, making it the most efficient choice for generating a ranked service portfolio. Where expert-derived weights are needed, AHP pairwise comparisons can serve as an upstream step before feeding the results into TOPSIS, a combination demonstrated by Ishak and Wanli (2020) in a regulated operational context.

Albarak *et al.* (2022) provide the most directly analogous precedent, combining TOPSIS with portfolio theory to prioritise technical debt and showing that it surfaced high-priority items likely to be deferred under effort-only approaches. TOPSIS has also been applied in other risk-intensive, regulated contexts (Zytoon, 2020; Khan and Purohit, 2022), providing methodological precedents for its use here. None of these studies, however, operates on live service management data in a large enterprise setting, which is a defining characteristic of the present research.

2.7 Transferability Critique and Ethical Dimensions

Although the preceding sections provide strong motivation for the proposed framework, none of the existing applications reviewed operates on live operational records in large enterprise settings. Most rely on researcher-defined scenarios, synthetic datasets or controlled development artefacts. The proposed research differs by applying TOPSIS to operational records such as incident logs, ITSM change data and vulnerability scan outputs, which are known to

have quality issues. Incident classifications can vary between teams. Recorded change failure rates can be influenced by documentation practices as much as by technical quality. Vulnerability counts depend on scanner coverage and configuration. These data limitations must be explicitly acknowledged. The sensitivity of rankings to input variation must be studied before outputs can be interpreted with confidence (Greco *et al.*, 2019).

Using operational service management data as a basis for technical debt prioritisation also raises important ethical questions. Such records are generated and owned by operations teams, and development teams may not have consented to their use in performance evaluations. A framework that ranks services based on operational indicators may influence budgets, staffing and reputational assessments in regulated institutions. These considerations are addressed in full in the methodology chapter.

2.8 Synthesis: Converging Evidence and the Research Gap

The four thematic sections together define the conceptual, methodological, contextual and technical landscape within which this research is situated.

2.8.1 Cross-Pillar Convergence

The preceding sections converge on a common conclusion: the current technical debt literature is conceptually rich but methodologically fragmented, and it remains poorly aligned with the operational realities of enterprise IT.

The first pillar concerns scope. Technical debt has expanded beyond source code to include architecture, infrastructure, vendor platforms, and operational dependencies (Yang *et al.*, 2019; Kleinwaks, Batchelor and Bradley, 2023), yet the dominant measurement approaches still concentrate on code-level artefacts (Biazotto *et al.*, 2024). This leaves important forms of debt, especially those that emerge in regulated and vendor-heavy environments, only partially observable (Rinta-Kahila *et al.*, 2023).

The second pillar concerns prioritisation. Existing studies show that technical debt is not a single dimension but a multi-criteria problem involving cost, value, risk, and operational impact (Lenarduzzi *et al.*, 2021; Alfayez *et al.*, 2023). However, the literature does not provide a stable quantitative method that combines these dimensions into a transparent ranking mechanism for service-level decision-making (Pina, Seaman and Goldman, 2022).

The third pillar concerns consequences. The empirical literature consistently shows that unmanaged technical debt produces measurable harm, including increased incidents, slower recovery, reduced productivity, higher remediation cost, and greater vulnerability exposure (Banker, Liang and Ramasubbu, 2020; de Toledo *et al.*, 2021; Tornhill and Borg, 2022). These effects are not abstract, they appear in operational outcomes that can be observed, compared, and prioritised (Ramasubbu and Kemerer, 2021).

The fourth pillar concerns measurement opportunity. Operational service data, including incident frequency, recovery time, and change failure rate, offers a practical way to observe debt related symptoms in environments where source code is inaccessible or incomplete. Patch recency can similarly be operationalised as an infrastructure lifecycle debt proxy using unsupported component data (Haki *et al.*, 2023), and delayed patching has been shown to produce measurable operational risk (Tizio, Armellini and Massacci, 2023). These indicators do not claim to measure technical debt directly. Rather, they serve as proxies for the operational consequences of debt accumulation and therefore provide a defensible basis for prioritisation in enterprise IT.

Taken together, these pillars reveal the research gap addressed by this dissertation. There is no validated service level framework that combines operational indicators into a single ranked view of technical debt priority in trading IT. The present study responds to that gap by developing and evaluating a TOPSIS based framework grounded in operational data and supported by sensitivity analysis (Hevner *et al.*, 2004).

2.8.2 Cross-Cutting Limitations

Several limitations qualify these conclusions. The empirical base for financial IT and trading systems remains relatively thin. Studies at Credit Suisse, at BankAlpha (a pseudonym) and at ING each provide valuable insights but are restricted to a single organisation (Kula *et al.*, 2019; Rolland and Lyytinen, 2021; Haki *et al.*, 2023). This scarcity of empirical evidence from trading IT environments specifically represents a gap that the present research is positioned to address, contributing an operational dataset from a regulated financial environment that is currently absent from the technical debt literature.

Existing applications of TOPSIS to technical debt rely on controlled datasets rather than live operational records (Albarak *et al.*, 2022; Khan and Purohit, 2022). The comparative work of Shekhovtsov and Saabun (2020) shows that method choice matters less than weight specification when weights are uncertain, which underscores the importance of transparent and justified weight determination over method selection *per se*.

Behavioural studies emphasise that management intent, incentives and communication patterns often exert a stronger influence on remediation outcomes than technical tools (Besker, 2020; Wiese and Borowa, 2023; Haki *et al.*, 2023). Technical frameworks can support but not replace governance and leadership.

2.8.3 The Research Gap

To the best of current literature, no framework integrates service-level operational metrics, multi-criteria prioritisation and the specific constraints of financial trading IT into a single, auditable technical debt ranking instrument. Code-centric tools are structurally inapplicable to vendor-hosted and regulated platforms. Single-criterion models conflict with both the empirical evidence on practitioner behaviour and the multidimensional nature of operational risk in trading environments. The proposed framework addresses this gap directly. Its evaluation will follow the approach established in adjacent quantitative work, assessing ranking stability across weighting schemes and validating indicator scores against historical operational records (Greco *et al.*, 2019; Albarak *et al.*, 2022).

3 Research Design and Methodology

This chapter explains the philosophical approach, research strategy, and design decisions that govern the construction and evaluation of the proposed framework. It proceeds in two parts: the first covers the research design from philosophical positioning through to the evaluation approach; the second addresses the ethical and professional considerations arising from the use of operational data in a regulated financial institution.

3.1 Research Design

This study is grounded in a pragmatist research philosophy. Pragmatism holds that the value of a knowledge claim is determined by whether the resulting artefact works in the context for which it was designed (Kelly and Cordeiro, 2020). This position suits the present study because the objective is not to establish universal laws about technical debt, but to construct a quantitative framework useful for prioritisation decisions in a specific enterprise IT environment. Technical debt is treated here as a construct whose meaning is shaped by measurement choices rather than as a directly observable quantity (Amanatidis *et al.*, 2020). The framework therefore grounds its estimates in operationally defined proxies and acknowledges that the resulting scores are representations, not facts.

3.1.1 Research Strategy

Design Science Research (DSR) governs this study, as formalised by Hevner *et al.* (2004). DSR is appropriate here because the primary output is an artefact rather

than a theory: a TOPSIS-based prioritisation framework operating on live ITSM data. The framework is designed to address the gap identified in Chapter 2, where no existing approach integrates service-level operational metrics for technical debt prioritisation in trading IT. Lenarduzzi *et al.* (2021) confirm the broader absence of validated, quantitative prioritisation frameworks across the field; the service-level and enterprise-IT dimension of that gap is established through the synthesis in Chapter 2.

Hevner *et al.* (2004) specify seven guidelines for rigorous DSR, mapped to this study in Table 3.1. On one side sits the problem space: the absence of a service-level TD framework. On the other sits the solution space: the TOPSIS artefact. These are connected through input knowledge, such as DORA metrics and MCDM methods, and through research processes, including indicator selection and sensitivity analysis. This structure positions the framework as prescriptive design knowledge applicable to regulated enterprise contexts.

TABLE 3.1: DSR guidelines and their implementation

DSR Guideline	Implementation in this Study
Design as an artefact	The TOPSIS-based prioritisation framework
Problem relevance	Established in Chapter 2: no existing framework integrates service-level operational metrics for TD prioritisation in trading IT
Design evaluation	Sensitivity analysis; criterion validity against historical operational records
Research contributions	Novel indicator set; first application of TOPSIS to live ITSM data in financial trading IT
Research rigour	Systematic indicator selection; reproducible TOPSIS computation; documented normalisation
Design as a search process	Iterative refinement of indicator selection (literature candidates audited against ITSM/CMDB availability) and weight assignment
Communication of research	This dissertation; practitioner-accessible scoring output

The process follows Vom Brocke, Hevner and Maedche (2020)'s six-step model: problem identification and motivation (Chapter 1 and Chapter 2), definition of objectives (Chapter 2), design and development of the artefact (Chapter 4), demonstration, evaluation (Chapter 5), and communication (Chapter 6). The sequence is not strictly linear; indicator selection and evaluation criteria were refined iteratively as constraints in the available data emerged during development.

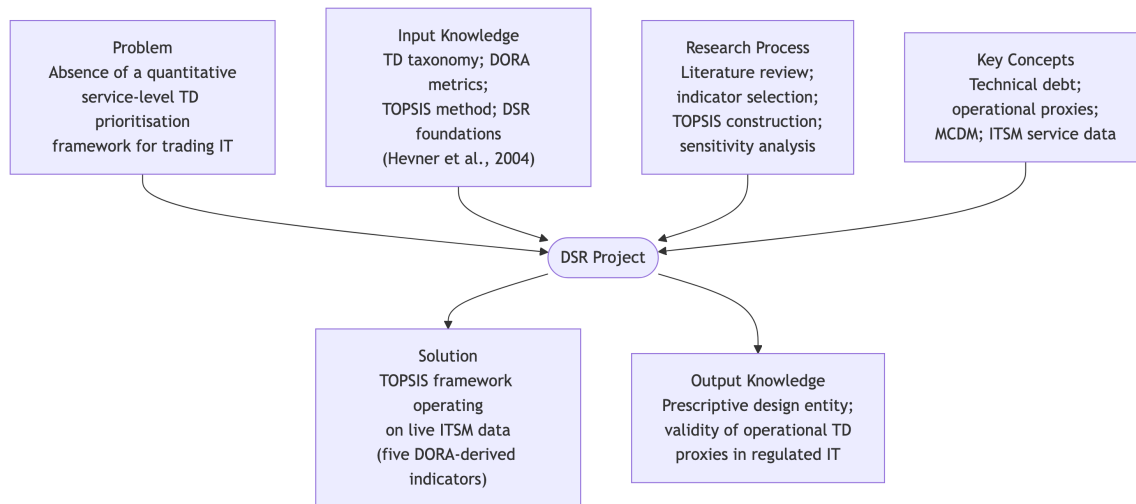


FIGURE 3.1: DSR Grid for the present study, adapted from vom Brocke, Hevner and Maedche (2020)

3.1.2 Research Design

The study is structured as a single-organisation embedded case study, situated within the trading IT division of one of Switzerland's three largest universal banks. Turnbull, Chugh and Luck (2021) define this design as an investigation into a single instance of observable complex phenomena, constrained by clearly identifiable boundaries. Discrete units of analysis, in this case individual IT services, are examined within that bounded organisational setting.

Three factors make this design appropriate. First, the regulatory, operational, and vendor-platform characteristics specific to Swiss financial IT resist generalisation from software engineering studies conducted elsewhere. Second, access to granular ITSM records is unavailable through surveys or experiments. Third, Hevner *et al.* (2004) specify that a design artefact must be evaluated through rigorous, well-executed methods. The present study satisfies this requirement

by evaluating the framework against operational data from the environment for which it was designed.

The study adopts the individual IT service as the unit of analysis, defined as a distinct configuration item in the organisation's ITSM instance. In this vendor-heavy enterprise environment, source-code access is unavailable for both outsourced components and on-premises vendor software (Ramasubbu and Kemerer, 2021). Service-level operational indicators are therefore tested as candidate proxies for technical debt prioritisation, rather than assumed as established substitutes. The study is cross-sectional, analysing a fixed observation window rather than tracking changes over time, consistent with the objective of demonstrating ranking validity at a point in time. Longitudinal evaluation is identified as a direction for future work.

3.1.3 Data Sources and Collection

The literature review (Chapter 2) identified candidate indicators mapping technical debt dimensions to operational data (see Table 2.3). A data audit against available ITSM and CMDB fields refined this set prior to data collection. Documentation completeness was excluded as no structured ITSM or CMDB field captures it consistently across services; relevant records reside in unstructured repositories not amenable to uniform extraction. Test coverage ratio and architectural coupling were excluded because both require access to assets unavailable in this environment: test coverage is derived from CI/CD build artefacts or source repositories, and architectural coupling is measured through static dependency graph analysis of compiled code (Amanatidis *et al.*, 2020); neither is accessible for vendor-managed nor outsourced components. Raw vulnerability density was excluded as subsumed by patch recency, which provides a more stable CMDB-derived proxy grounded in regulatory patching compliance rather than raw scanner counts whose coverage varies across components.

The five retained indicators are fully operationalisable from live records held in the organisation's ITSM instance and CMDB. Records were accessed under an agreed arrangement with the Head of Trading IT over a twelve-month observation window (January–December 2025) and anonymised at source, with

service names replaced by alphanumeric identifiers and all team and vendor labels removed before analysis.

TABLE 3.2: Operational definition of indicators used in the TOPSIS framework

Indicator	Operational Definition	Source	Unit	Direction
Incident frequency	Count of P2 and P3 incidents per service within the observation window, normalised by service age in months. P1 incidents, defined by the organisation as cases where the majority of its business is affected, did not occur within the observation period and are excluded.	ITSM	Incidents / month	Cost
MTTR	Mean elapsed time from incident creation to resolution status, averaged across all P2 and P3 incidents per service	ITSM	Hours	Cost
CFR	Proportion of change records per service that resulted in an unplanned incident or rollback within 72 hours of deployment	ITSM	Percent-age (0–100)	Cost
Patch recency	Proportion of components within each service's dependency stack recorded as within a vendor-supported version window at point of extraction	CMDB	Percent-age (0–100)	Benefit

Indicator	Operational Definition	Source	Unit	Direction
Unsupported component months	Cumulative months of exposure to			
unsupported components within the service's CMDB dependency stack during the observation window	CMDB	Months	Cost	

Incident frequency, MTTR and CFR proxy the operational consequences of architectural and infrastructure debt. Service instability and slow recovery are empirically linked to TD severity in vendor-managed portfolios (Ramasubbu and Kemerer, 2021; Nielsen and Skaarup, 2022). Deployment instability is classified as a direct observable symptom of accumulated debt (Forsgren, Humble and Kim, 2018; Nord, Ozkaya and Woody, 2021). Nord, Ozkaya and Woody (2021) further ground the inclusion of MTTR and CFR specifically by classifying delivery tempo degradation as a diagnostic TD signal detectable from production records, independently of source code access. Patch recency captures the proportion of components currently within a vendor-supported version window. Unsupported component months measures the cumulative duration of exposure to end-of-life components during the observation period. Together, these two CMDB-derived indicators operationalise infrastructure lifecycle and vulnerability debt following Haki *et al.* (2023), differentiating services with similar current patch status but differing historical debt accumulation. Service-level operational indicators are treated as candidate proxies for technical debt prioritisation rather than established substitutes. The validity of this inference is the central empirical hypothesis tested in Chapter 5.

3.1.4 Framework Design

The analytical core is the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS), formulated by Hwang and Yoon (1981) and validated through comprehensive reviews of its enterprise applications (Pandey, Komal and Dincer, 2023). TOPSIS addresses the single-criterion prioritisation limitation identified in Chapter 2 by ranking services according to their geometric proximity to ideal solutions after normalisation and weighting. Given a decision matrix with m services and n indicators, the method proceeds as follows.

The matrix is normalised using vector normalisation, which scales each value relative to the column norm to ensure comparability across indicators with different units and ranges (Pandey, Komal and Dincer, 2023):

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^m x_{ij}^2}}$$

A weighted normalised matrix is constructed using criterion weight w_j :

$$v_{ij} = w_j r_{ij}$$

The positive and negative ideal solutions are identified:

$$A^+ = \{v_1^+, \dots, v_n^+\}, \quad A^- = \{v_1^-, \dots, v_n^-\}$$

Euclidean distances from each ideal are calculated:

$$S_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^+)^2}, \quad S_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^-)^2}$$

The relative closeness coefficient C_i constitutes the final score:

$$C_i = \frac{S_i^-}{S_i^+ + S_i^-}, \quad C_i \in [0, 1]$$

A higher C_i indicates a service closer to the worst-case operational profile, representing a higher technical debt priority. Incident frequency, MTTR, and CFR are cost criteria where lower values are preferable; patch recency is a benefit criterion where higher values are preferable.

Figure 3.2 illustrates the full pipeline from indicator collection through to ranked output.

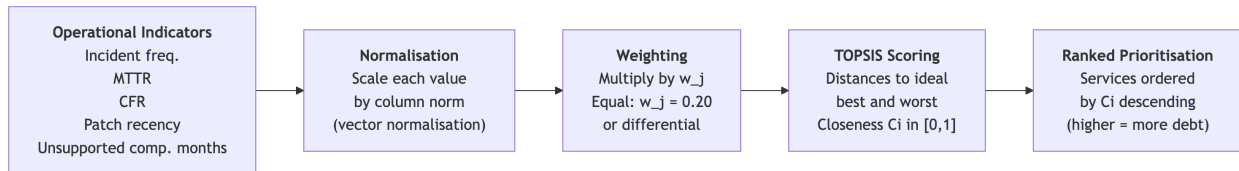


FIGURE 3.2: Framework pipeline: from operational indicators to ranked prioritisation

Two weight configurations are used. A baseline equal-weight configuration ($w_j = 0.25$) serves as the reference case, in accordance with Greco *et al.* (2019). A secondary configuration applies differential weights reflecting the higher operational salience of MTTR and CFR in a trading environment, as established in Chapter 2. The sensitivity of rankings to weight perturbation is assessed in Chapter 5.

3.1.5 Evaluation Approach

The framework is evaluated against a single criterion following the sensitivity practice recommended by Greco *et al.* (2019) for composite indicators. Ranking stability is assessed by perturbing each criterion weight by plus or minus 20 percentage points, with remaining weights adjusted proportionally, and measuring rank-order correlations between the baseline ranking and each perturbed variant using Kendall's τ . This constitutes an analytical evaluation of the artefact in the sense of Hevner *et al.* (2004) Guideline 3, which requires that the utility and quality of a design artefact be rigorously demonstrated via well-executed evaluation methods. A consistently high τ across perturbations provides evidence that the ranking is not sensitive to reasonable variation in weight assumptions, which is a necessary condition for the framework to be actionable in practice.

3.2 Ethical and Professional Considerations

The ethical considerations arising from this study concern data confidentiality and legal compliance, the responsible use of prioritisation outputs, and alignment with professional standards.

3.2.1 Data Confidentiality and Anonymisation

The data originate from a live Swiss financial institution that is not identified by name. All service, team, and vendor identifiers were replaced with alphanumeric codes before analysis. No employee data are present: all five indicators are service-level aggregates that cannot be attributed to individuals. The data-sharing arrangement was established with the explicit agreement of the Head of Trading IT, access was limited to the required records, and no data were retained beyond the analytical period or shared with third parties.

3.2.2 Legal Compliance

The organisation is subject to the revised Federal Act on Data Protection (nFADP), which entered into force on 1 September 2023 (*Federal Act on Data Protection*, 2023), enforced by the Federal Data Protection and Information Commissioner (FDPIC). Because the dataset contains no personal data, the nFADP's provisions on personal data processing do not apply directly. The principle of data minimisation, which the nFADP enshrines in alignment with international standards, was nonetheless observed: only the metrics required for the five TOPSIS indicators were extracted.

As a systemically significant institution, the organisation is also subject to FINMA Circular 2023/1 on operational risks and resilience, in force since 1 January 2024 (*FINMA Circular 2023/1 - Operational risks and resilience – banks*, 2024). This circular requires comprehensive ICT operational risk management, including incident monitoring and service resilience. Areas structurally aligned with the framework's operational focus. Although the specific metrics used here are organisationally defined rather than prescribed by FINMA. No ethics committee approval was required under the University of Essex's research ethics framework, and a departmental no-objection confirmation was obtained.

3.2.3 Responsible Use of Prioritisation Outputs

A relative rank assigned to a service carries a risk of being misused to evaluate team performance. High-debt service profiles typically reflect historical design decisions or legacy constraints rather than the competence of the team currently managing the service (Ahmad *et al.*, 2026). Three mitigations address this risk: framework documentation explicitly states that C_i reflects the operational state of a service rather than team performance; outputs are intended for IT governance and portfolio management functions only; and the sensitivity analysis makes weighting assumptions transparent, reducing the risk of false precision.

3.2.4 Professional Standards and Limitations

This study was conducted in accordance with the BCS Code of Conduct (*Code of Conduct for BCS Members*, 2022), including the obligations to act with integrity, to respect the rights of third parties, and to ensure that the artefact is fit for purpose. Three limitations clarify scope. The single-site design means that indicator weights and TOPSIS configuration are calibrated for one organisation and should not be transferred without revalidation. The cross-sectional window means the framework captures a snapshot; recently remediated services may be under-ranked. Finally, the five indicators do not exhaust all dimensions of operational technical debt; indicators related to documentation, test coverage, or architectural coupling were excluded due to data availability and remain candidates for future iterations.

3.3 Rework!

The following diagram illustrates the logical pipeline of the proposed framework, from the collection of operational indicators through to a ranked prioritisation output.



FIGURE 3.3: Proposed framework pipeline

4 Framework Development and Results

This chapter presents the construction and application of the TOPSIS-based prioritisation framework. It covers data preparation, indicator normalisation and weight assignment, TOPSIS computation and ranking, and sensitivity analysis, before discussing the substantive interpretation of the results. The complete Python implementation is provided in Appendix A. All code referenced in this chapter can be executed directly to reproduce the outputs shown here.

4.1 Data Preparation

The dataset covers five IT services drawn from the trading IT division of the case organisation, extracted from its ITSM instance. Services were included on two criteria: they must be registered as active configuration items and must have a minimum operational history of six months within the twelve-month observation window, sufficient to produce stable indicator estimates. Services with fewer than three recorded production changes during the window were assigned the portfolio mean for CFR, consistent with the imputation approach recommended for sparse operational datasets (Greco *et al.*, 2019). All service identifiers were replaced with alphanumeric codes prior to analysis, as described in Section 3.2.1. No team, vendor or individual identifiers are present in the dataset.

The four indicators are operationally defined as follows, consistent with the definitions established in Section 3.1.4. Incident frequency is the total number of P2 and P3 incidents attributed to the service during the observation window, divided by service age in months. MTTR is the arithmetic mean of elapsed

hours between incident creation and resolution across all P2 and P3 incidents within the window. CFR is the proportion of production changes that resulted in an unplanned rollback or a subsequent incident within 72 hours of deployment. Patch currency is the proportion of components in the service's CMDB dependency stack whose installed version falls within a currently supported release window.

TABLE 4.1: Raw decision matrix

Service	Incident Freq.	MTTR (hrs)	CFR	Patch Recency
SVC-01	1.8	38.4	0.27	0.52
SVC-02	0.9	15.2	0.12	0.75
SVC-03	2.4	47.1	0.33	0.46
SVC-04	0.3	5.8	0.03	0.94
SVC-05	0.4	44.2	0.05	0.89

Incident frequency, MTTR and CFR are cost criteria, for which lower values indicate lower technical debt severity. Patch currency is a benefit criterion, for which higher values are preferable.

4.1.1 Data Limitations

As established in Section 2.7, the operational data used here carry inherent quality limitations that must be explicitly acknowledged. Incident P2/P3 classifications vary across teams; CFR reflects documentation practices as much as technical outcomes; and patch recency depends on CMDB coverage and scanner configuration. These do not invalidate the proxy approach but constrain score precision.

Mitigations include: (i) sensitivity analysis (Section 4.4) to test ranking stability; and (ii) cross-validation against independent remediation records (Chapter 5).

4.2 Indicator Normalisation and Weighting

Before aggregation, each indicator is normalised using vector normalisation to remove differences in scale and unit:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^m x_{ij}^2}}$$

Vector normalisation is preferred over min-max normalisation because it preserves proportional relationships between values without compressing the range to a fixed interval, a property that (Greco *et al.*, 2019) recommend for composite indicators where extreme values carry substantive meaning. A service with an unusually high MTTR should retain its full distance from lower-performing services in the normalised space rather than be pulled toward a truncated maximum. A weighted normalised matrix is then constructed by multiplying each normalised value by its criterion weight:

$$v_{ij} = w_j r_{ij}$$

Two weight configurations are applied. The baseline equal-weight configuration assigns $w_j = 0.25$ to each criterion and serves as the reference case against which the differential configuration is evaluated. The differential configuration assigns higher weights to MTTR and CFR.

This reflects a design judgement rather than a claim attributable to a single empirical source, and it is grounded in three converging observations from the literature. First, Ramasubbu, Kemerer and Woodard (2015) establish that technical debt in COTS enterprise systems increases the probability of system failures by up to 62 percent, identifying incident frequency as the most directly observable TD signal in vendor-platform environments. This makes the speed of recovery from those incidents, captured by MTTR, the dimension most consequential to operational governance once a failure has occurred. Second, Nord, Ozkaya and Woody (2021) classify delivery tempo degradation and deployment instability explicitly as observable symptoms from which technical debt can be detected and prioritised in production environments (Nord, Ozkaya and Woody,

2021). This provides direct grounding for elevating the weights of both MTTR and CFR relative to the remaining two criteria. Third, Fungprasertkul, Bahsoon and Kazman (2024) establish that TD dimensions can and should be assigned differential weights to reflect the perspectives of different stakeholders and the operational context in which monitoring takes place (Fungprasertkul, Bahsoon and Kazman, 2024), offering methodological precedent for a two-configuration design in which the baseline equal-weight case is tested against a configuration calibrated to domain context.

Two weight configurations are applied, as specified in Section 3.1.4. The baseline equal-weight configuration assigns $w_j = 0.25$ to each criterion and serves as the reference case. The differential configuration assigns higher weights to MTTR and CFR, reflecting their higher operational salience in a trading environment where speed of recovery and deployment stability directly determine financial exposure (Forsgren, Humble and Kim, 2018). The differential weights are $w_{\text{MTTR}} = w_{\text{CFR}} = 0.30$ and $w_{\text{IncFreq}} = w_{\text{Patch}} = 0.20$.

TABLE 4.2: Weight configurations

Configuration	Incident			Patch
	Freq.	MTTR	CFR	Currency
Equal (baseline)	0.25	0.25	0.25	0.25
Differential	0.20	0.30	0.30	0.20

4.3 TOPSIS Computation and Rankings

The TOPSIS procedure follows the formulation of (Hwang and Yoon, 1981). After constructing the weighted normalised matrix, the positive and negative ideal solutions are:

4.4 Sensitivity Analysis

4.5 Interpretation and Discussion

5 Evaluation

5.1 Evaluation Aim

5.2 Evaluation Criteria

5.3 Stability Results

5.4 Practical Validity

5.5 Limitations of the Evaluation

6 Conclusion

6.1 Research Summary

6.2 Findings Against Research Questions

6.3 Contribution to Knowledge

6.4 Practical Contribution

6.5 Limitations and Future Work

6.6 Reflective Note

References

Ahmad, M.O., Mandi, V., Tauan, N., Katin, A. and Herath, P. (2026) 'Technical debt is not just technical: An industrial case study in large agile software development', *Journal of Systems and Software*, 234, p. 112719. Available at: <https://doi.org/10.1016/j.jss.2025.112719>.

Albarak, M., Bahsoon, R., Ozkaya, I. and Nord, R.L. (2022) 'Managing Technical Debt in Database Normalization', *IEEE Transactions on Software Engineering*, 48(3), pp. 755–772. Available at: <https://doi.org/10.1109/TSE.2020.3001339>.

Albuquerque, D., Guimarães, E., Tonin, G., Rodríguez, P., Perkusich, M., Almeida, H., Perkusich, A. and Chagas, F. (2023) 'Managing Technical Debt Using Intelligent Techniques - A Systematic Mapping Study', *IEEE Transactions on Software Engineering*, 49(4), pp. 2202–2220. Available at: <https://doi.org/10.1109/TSE.2022.3214764>.

Alfayez, R., Winn, R., Alwehaibi, W., Venson, E. and Boehm, B. (2023) 'How SonarQube-identified technical debt is prioritized: An exploratory case study', *Information and Software Technology*, 156, p. 107147. Available at: <https://doi.org/10.1016/j.infsof.2023.107147>.

Amanatidis, T., Mittas, N., Moschou, A., Chatzigeorgiou, A., Ampatzoglou, A. and Angelis, L. (2020) 'Evaluating the agreement among technical debt measurement tools: Building an empirical benchmark of technical debt liabilities', *Empirical Software Engineering*, 25(5), pp. 4161–4204. Available at: <https://doi.org/10.1007/s10664-020-09869-w>.

Ampatzoglou, A., Chatzigeorgiou, A., Arvanitou, E.M. and Bibi, S. (2022)

'SDK4ED: A platform for technical debt management', *Software: Practice and Experience*, 52(8), pp. 1879–1902. Available at: <https://doi.org/10.1002/spe.3093>.

Avgeriou, P., Ozkaya, I., Chatzigeorgiou, A., Ciolkowski, M., Ernst, N.A., Koontz, R.J., Poort, E. and Shull, F. (2023) 'Technical Debt Management: The Road Ahead for Successful Software Delivery', *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, Melbourne, Australia: IEEE, pp. 15–30. Available at: <https://doi.org/10.1109/ICSE-FoSE59343.2023.00007>.

Banker, R., Liang, Y. and Ramasubbu, N. (2020) 'Technical Debt and Firm Performance', *Management Science*, 67(5), pp. 3174–3194. Available at: <https://doi.org/10.1287/mnsc.2019.3542>.

Bedi, J. and Kaur, K. (2022) 'Understanding factors affecting technical debt', *International Journal of Information Technology*, 14(2), pp. 1051–1060. Available at: <https://doi.org/10.1007/s41870-020-00487-9>.

Besker, T. (2020) *Technical Debt: An empirical investigation of its harmfulness and on management strategies in industry*. Chalmers University of Technology. Available at: https://research.chalmers.se/publication/518318/file/518318_Fulltext.pdf.

Biazotto, J.P., Feitosa, D., Avgeriou, P. and Nakagawa, E.Y. (2024) 'Technical debt management automation: State of the art and future perspectives', *Information and Software Technology*, 167, p. 107375. Available at: <https://doi.org/10.1016/j.infsof.2023.107375>.

Ciardiello, F. and Genovese, A. (2023) 'A comparison between TOPSIS and SAW methods', *Annals of Operations Research*, 325(2), pp. 967–994. Available at: <https://doi.org/10.1007/s10479-023-05339-w>.

Code of Conduct for BCS Members (2022). Swindon, UK: BCS, The Chartered Institute for IT, pp. 1–5. Available at: <https://www.bcs.org/media/2211/bcs->

[code-of-conduct.pdf](#) (Accessed: April 15, 2026).

Cunningham, W. (1992) 'The WyCash portfolio management system', *ACM SIGPLAN OOPS Messenger*, 4(2), pp. 29–30. Available at: <https://doi.org/10.1145/157710.157715>.

Federal Act on Data Protection (2023). Available at: <https://www.fedlex.admin.ch/eli/cc/2022/491/en> (Accessed: April 15, 2026).

FINMA Circular 2023/1 - Operational risks and resilience – banks (2024). Available at: <https://www.finma.ch/en/~media/finma/dokumente/dokumentencenter/myfinma/rundschreiben/finma-rs-2023-01-20221207.pdf> (Accessed: March 29, 2026).

Forsgren, N., Humble, J. and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR: IT Revolution Press.

Fowler, M. (2009) *Technical Debt Quadrant*. Technical Debt Quadrant. Available at: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html> (Accessed: March 29, 2026).

Fungprasertkul, S., Bahsoon, R. and Kazman, R. (2024) 'Technical Debt Monitoring Decision Making with Skin in the Game', *ACM Transactions on Software Engineering and Methodology*, 33(7), pp. 1–27. Available at: <https://doi.org/10.1145/3664805>.

García-Cascales, M.S. and Lamata, M.T. (2012) 'On rank reversal and TOPSIS method', *Mathematical and Computer Modelling*, 56(5), pp. 123–132. Available at: <https://doi.org/10.1016/j.mcm.2011.12.022>.

Greco, S., Ishizaka, A., Tasiou, M. and Torrisi, G. (2019) 'On the Methodological Framework of Composite Indices: A Review of the Issues of Weighting, Aggregation, and Robustness', *Social Indicators Research*, 141(1), pp. 61–94. Available

at: <https://doi.org/10.1007/s11205-017-1832-9>.

Haki, K., Rieder, A., Buchmann, L. and W.Schneider, A. (2023) 'Digital nudging for technical debt management at Credit Suisse', *European Journal of Information Systems*, 32(1), pp. 64–80. Available at: <https://doi.org/10.1080/0960085X.2022.2088413>.

Hanssen, G.K., Brataas, G. and Martini, A. (2019) 'Identifying Scalability Debt in Open Systems', *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*. *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 48–52. Available at: <https://doi.org/10.1109/TechDebt.2019.00014>.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design Science in Information Systems Research', *MIS Quarterly*, 28(1), pp. 75–105. Available at: <https://doi.org/10.2307/25148625>.

Hwang, C.-L. and Yoon, K. (1981) *Multiple Attribute Decision Making*. Edited by M. Beckmann and H.P. Kunzi. Berlin, Heidelberg: Springer (Lecture Notes in Economics and Mathematical Systems). Available at: <https://doi.org/10.1007/978-3-642-48318-9>.

Ishak, A. and Wanli (2020) 'Analysis of Fuzzy AHP-TOPSIS Methods in Multi Criteria Decision Making: Literature Review', *IOP Conference Series: Materials Science and Engineering*, 1003(1), p. 012147. Available at: <https://doi.org/10.1088/1757-899X/1003/1/012147>.

Kelly, L.M. and Cordeiro, M. (2020) 'Three principles of pragmatism for research on organizational processes', *Methodological Innovations*, 13(2), p. 2059799120937242. Available at: <https://doi.org/10.1177/2059799120937242>.

Khan, S. and Purohit, L. (2022) 'An Integrated Methodology of Ranking Based on PROMETHEE-CRITIC and TOPSIS-CRITIC In Web Service Domain', *2022 IEEE 11th International Conference on Communication Systems and Network Technologies (CSNT)*. *2022 IEEE 11th International Conference on Communication Systems and*

Network Technologies (CSNT), pp. 335–340. Available at: <https://doi.org/10.1109/CSNT54456.2022.9787620>.

Kleinwaks, H., Batchelor, A. and Bradley, T.H. (2023) 'Technical debt in systems engineering—A systematic literature review', *Systems Engineering*, 26(5), pp. 675–687. Available at: <https://doi.org/10.1002/sys.21681>.

Kruchten, P., Nord, R.L. and Ozkaya, I. (2012) 'Technical Debt: From Metaphor to Theory and Practice', *IEEE Software*, 29(6), pp. 18–21. Available at: <https://doi.org/10.1109/MS.2012.167>.

Kula, E., Rastogi, A., Huijgens, H., Deursen, A. van and Gousios, G. (2019) 'Releasing fast and slow: An exploratory case study at ING', *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. New York, NY, USA: Association for Computing Machinery (ESEC/FSE 2019), pp. 785–795. Available at: <https://doi.org/10.1145/3338906.3338978>.

Lenarduzzi, V., Besker, T., Taibi, D., Martini, A. and Arcelli Fontana, F. (2021) 'A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools', *Journal of Systems and Software*, 171, p. 110827. Available at: <https://doi.org/10.1016/j.jss.2020.110827>.

Markkanen, V. and Frantti, T. (2023) 'Patch management planning - towards one-to-one policy', *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*. *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*, pp. 60–69. Available at: <https://doi.org/10.1109/DSA59317.2023.00018>.

McConnell, S. (2008) 'Managing Technical Debt'. Construx Software. Available at: https://www.construx.com/wp-content/uploads/2019/02/CxWhitePaper_TechnicalDebt.pdf (Accessed: March 21, 2026).

Nielsen, M.E. and Skaarup, S. (2022) 'IT Portfolio management as a framework for managing Technical Debt: Theoretical framework applied on a case study', *14th International Conference on Theory and Practice of Electronic Governance. ICEGOV 2021: 14th International Conference on Theory and Practice of Electronic Governance*, New York, NY, USA: Association for Computing Machinery, pp. 89–96. Available at: <https://doi.org/10.1145/3494193.3494296>.

Nord, R.L., Ozkaya, I. and Woody, C. (2021) *Examples of Technical Debt's Cybersecurity Impact*. Pittsburgh, PA, USA: Carnegie Mellon University, pp. 1–21. Available at: <https://apps.dtic.mil/sti/trecms/pdf/AD1144728.pdf> (Accessed: April 27, 2026).

Nurse, J.R.C. (2025) 'To Patch or Not to Patch: Motivations, Challenges, and Implications for Cybersecurity', in A. Moallem (ed.) *HCI for Cybersecurity, Privacy and Trust*. Cham: Springer Nature Switzerland, pp. 265–281. Available at: https://doi.org/10.1007/978-3-031-92833-8_16.

Pandey, V., Komal and Dincer, H. (2023) 'A review on TOPSIS method and its extensions for different applications with recent development', *Soft Computing*, 27(23), pp. 18011–18039. Available at: <https://doi.org/10.1007/s00500-023-09011-0>.

Paudel, B., Gonzalez-Huerta, J., Zabardast, E. and Klotins, E. (2025) 'Exploring the Relationship Between Technical Debt and Lead Time: An Industrial Case Study', *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 693–703. Available at: <https://doi.org/10.1109/SANER64311.2025.00071>.

Pina, D., Seaman, C. and Goldman, A. (2022) 'Technical Debt Prioritization: A Developer's Perspective', *2022 IEEE/ACM International Conference on Technical Debt (TechDebt). 2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 46–55. Available at: <https://doi.org/10.1145/3524843.3528096>.

Ramasubbu, N. and Kemerer, C.F. (2021) 'Controlling Technical Debt Remediation in Outsourced Enterprise Systems Maintenance: An Empirical Analysis', *Journal of Management Information Systems*, 38(1), pp. 4–28. Available at: <https://doi.org/10.1080/07421222.2021.1870377>.

Ramasubbu, N., Kemerer, C.F. and Woodard, C.J. (2015) 'Managing Technical Debt: Insights from Recent Empirical Evidence', *IEEE Software*, 32(2), pp. 22–25. Available at: <https://doi.org/10.1109/MS.2015.45>.

Rinta-Kahila, T., Penttinen, E., Aalto University, Lyytinen, K. and Case Western Reserve University (2023) 'Getting Trapped in Technical Debt: Sociotechnical Analysis of a Legacy System's Replacement', *MIS Quarterly*, 47(1), pp. 1–32. Available at: <https://doi.org/10.25300/MISQ/2022/16711>.

Rolland, K.-H. and Lyytinen, K. (2021) 'Exploring the Tensions Between Management of Architectural Debt and Digital Innovation: The Case of a Financial Organization', *Proceedings of the 54th Hawaii International Conference on System Sciences*. 54th Hawaii International Conference on System Sciences, Hawaii International Conference on System Sciences, pp. 6722–6732. Available at: <https://doi.org/10.24251/HICSS.2021.807>.

Shekhovtsov, A. and Saabun, W. (2020) 'A comparative case study of the VIKOR and TOPSIS rankings similarity', *Procedia Computer Science*, 176, pp. 3730–3740. Available at: <https://doi.org/10.1016/j.procs.2020.09.014>.

Stochel, M.G., Borek, T., Wawrowski, M.R. and Choda, P. (2023) 'Business-driven technical debt management using Continuous Debt Valuation Approach (CoDVA)', *Information and Software Technology*, 164, p. 107333. Available at: <https://doi.org/10.1016/j.infsof.2023.107333>.

Tizio, G.D., Armellini, M. and Massacci, F. (2023) 'Software Updates Strategies: A Quantitative Evaluation Against Advanced Persistent Threats', *IEEE Transactions on Software Engineering*, 49(3), pp. 1359–1373. Available at: <https://doi.org/10.1109/TSE.2022.3176674>.

de Toledo, S.S., Martini, A., Sjøberg, D.I.K., Przybyszewska, A. and Frandsen, J.S. (2021) 'Reducing Incidents in Microservices by Repaying Architectural Technical Debt', *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 196–205. Available at: <https://doi.org/10.1109/SEAA53835.2021.00033>.

Tornhill, A. and Borg, M. (2022) 'Code Red: The Business Impact of Code Quality - A Quantitative Study of 39 Proprietary Production Codebases', *2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*. *2022 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 11–20. Available at: <https://doi.org/10.1145/3524843.3528091>.

Turnbull, D., Chugh, R. and Luck, J. (2021) 'The Use of Case Study Design in Learning Management System Research: A Label of Convenience?', *International Journal of Qualitative Methods*, 20, pp. 1–11. Available at: <https://doi.org/10.1177/16094069211004148>.

Vidoni, M., Codabux, Z. and Fard, F.H. (2022) 'Infinite technical debt', *Journal of Systems and Software*, 190, p. 111336. Available at: <https://doi.org/10.1016/j.jss.2022.111336>.

Vom Brocke, J., Hevner, A. and Maedche, A. (2020) 'Introduction to Design Science Research', in J. Vom Brocke, A. Hevner, and A. Maedche (eds.) *Design Science Research. Cases*. Cham: Springer International Publishing, pp. 1–13. Available at: https://doi.org/10.1007/978-3-030-46781-4_1.

Wiese, M. and Borowa, K. (2023) 'IT managers' perspective on Technical Debt Management', *Journal of Systems and Software*, 202, p. 111700. Available at: <https://doi.org/10.1016/j.jss.2023.111700>.

Yang, W. (2020) 'Ingenuous Solution for the Rank Reversal Problem of TOPSIS Method', *Mathematical Problems in Engineering*, 2020(1), p. 9676518. Available at: <https://doi.org/10.1155/2020/9676518>.

Yang, Y., Michel, R., Wade, J., Verma, D., Törngren, M. and Alelyani, T. (2019) 'Towards a taxonomy of technical debt for COTS-intensive cyber physical systems', *Procedia Computer Science*, 153, pp. 108–117. Available at: <https://doi.org/10.1016/j.procs.2019.05.061>.

Zanakis, S.H., Solomon, A., Wishart, N. and Dublisch, S. (1998) 'Multi-attribute decision making: A simulation comparison of select methods', *European Journal of Operational Research*, 107(3), pp. 507–529. Available at: [https://doi.org/10.1016/S0377-2217\(97\)00147-1](https://doi.org/10.1016/S0377-2217(97)00147-1).

Zytoon, M.A. (2020) 'A Decision Support Model for Prioritization of Regulated Safety Inspections Using Integrated Delphi, AHP and Double-Hierarchical TOPSIS Approach', *IEEE Access*, 8, pp. 83444–83464. Available at: <https://doi.org/10.1109/ACCESS.2020.2991179>.

A Appendix A

Lorem ipsum